Synopsys Design Flow Tutorial

Professor: Sci.D., Professor
Vazgen Melikyan

Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

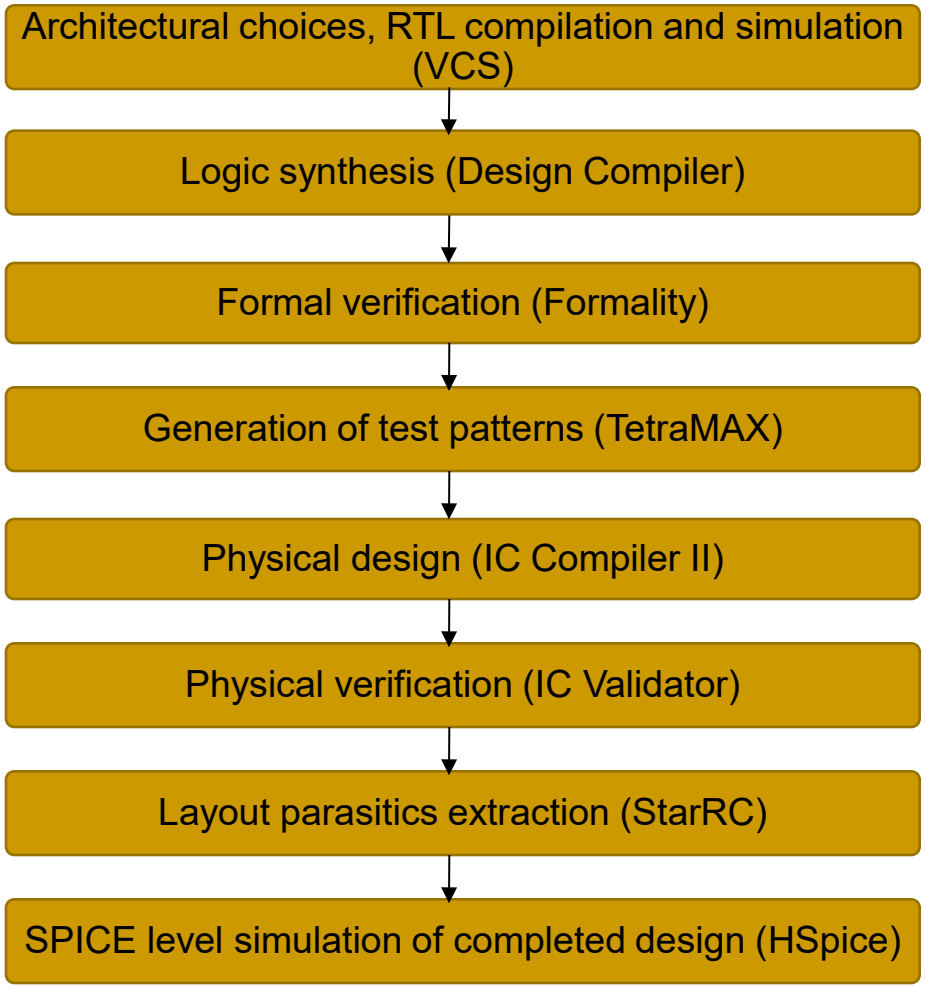
Course Overview

- Synopsys Design Flow
  - 2 lectures
- Logic Simulation (VCS)
  - 2 lectures
- Logic Synthesis (Design Compiler)
  - 2 lectures
- Physical Synthesis (IC Compiler II)
  - 2 lectures
- Static Timing Analysis (PrimeTime)
  - 2 lectures
- Formal Verification (Formality)
  - 2 lectures
- Automatic Test Pattern Generation (TetraMAX)
  - 2 lectures
- Physical Verification (IC Validator)
  - 2 lectures
- Layout Parasitics Extraction (StarRC)
  - 2 lectures
- SPICE-Level Simulation of Completed Design (HSpice)
  - 2 lectures

Synopsys Design Flow

Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

Synopsys Design Flow



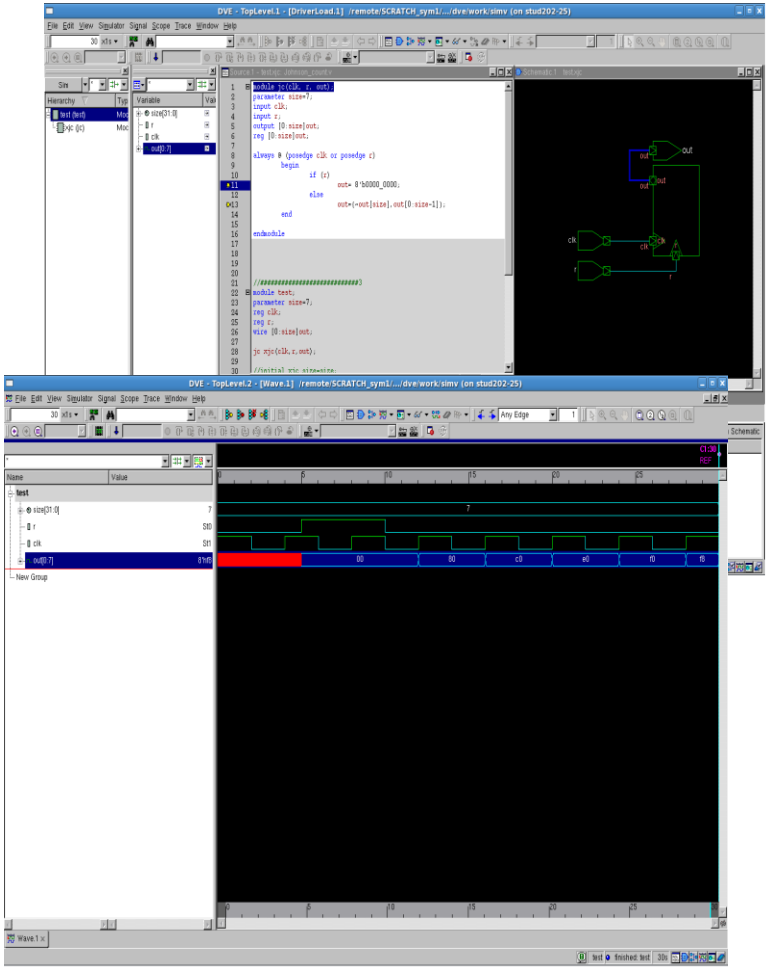
Logic Simulation (VCS)

VCS Overview

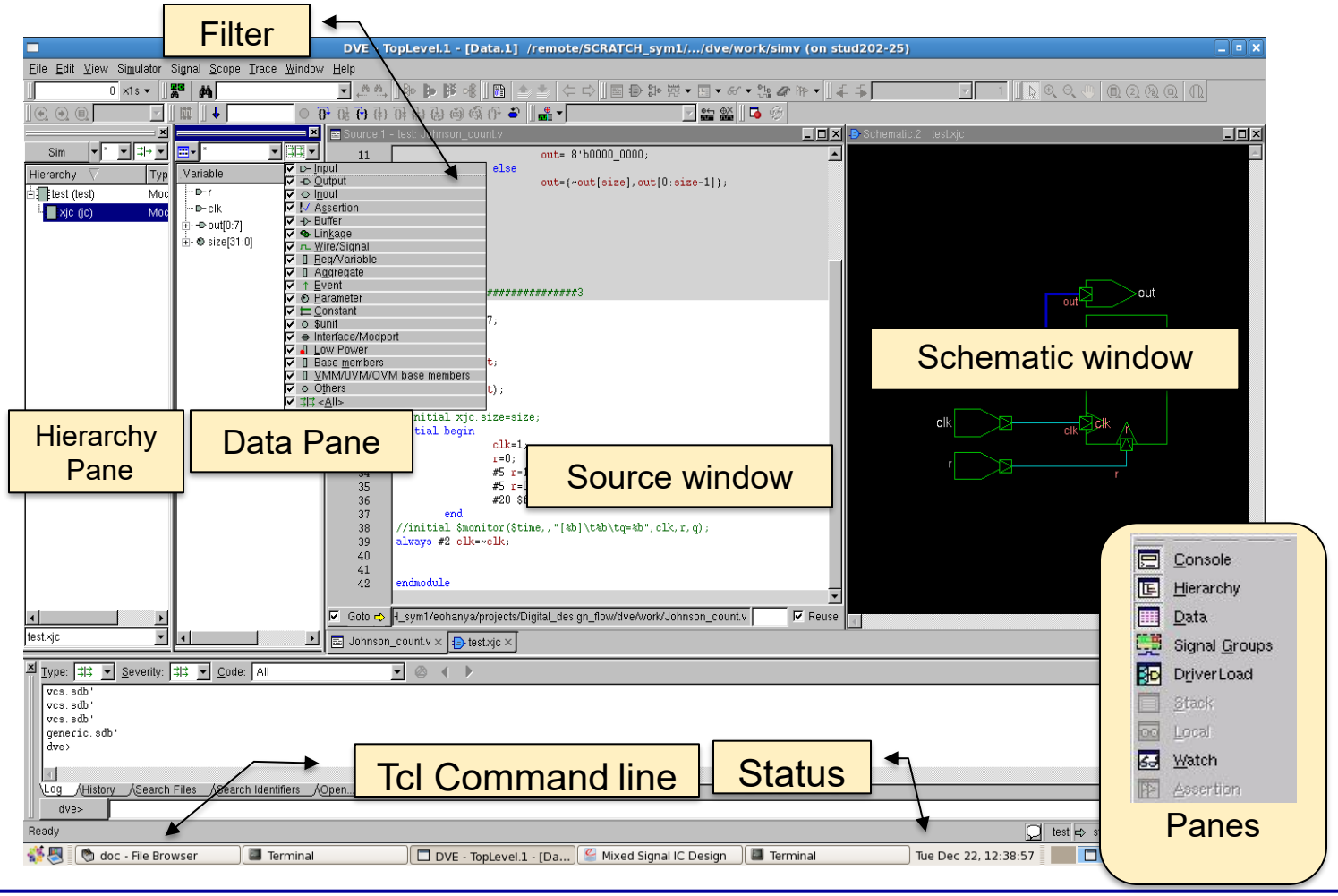
- VCS supports multiple languages
  - Verilog
  - VHDL
  - C/C++
  - SystemC
  - SystemVerilog
  - OpenVera
  - Analog
- Intuitive GUI help find bugs quickly
  - Assertions
  - Testbench
  - Coverage
  - Post-simulation analysis

VCS Features

- The most used features are:
  - Tracing the Cause of Failed Assertion
  - Trace drivers and loads of a signal at any time to see the drivers and loads that caused a value change and see all the drivers/loads that possibly contributed to a signal value.
  - RTL and Gate Signal Comparison
  - Highlighting the net in gate-level schematic and Verilog

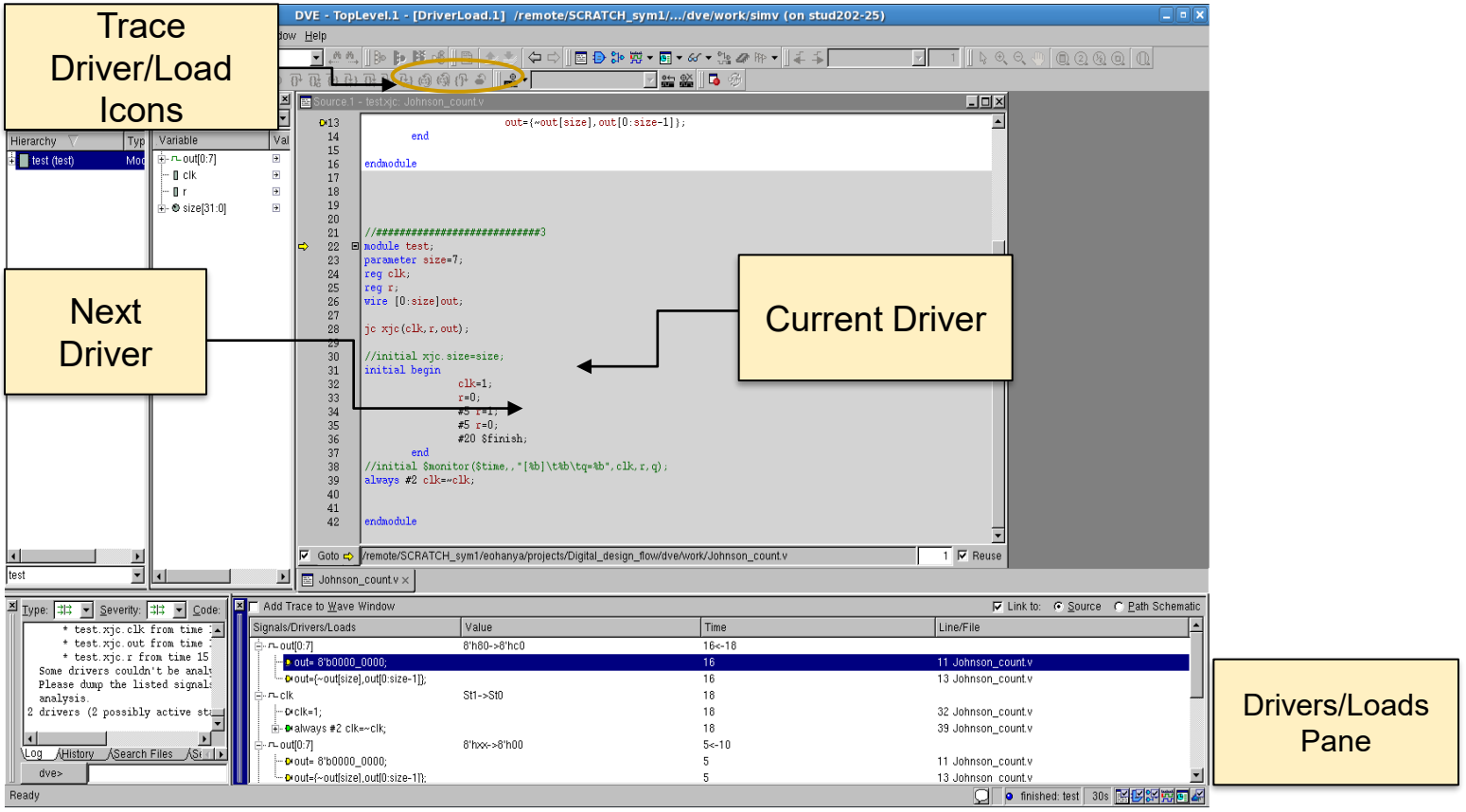


DVE TopLevel Window

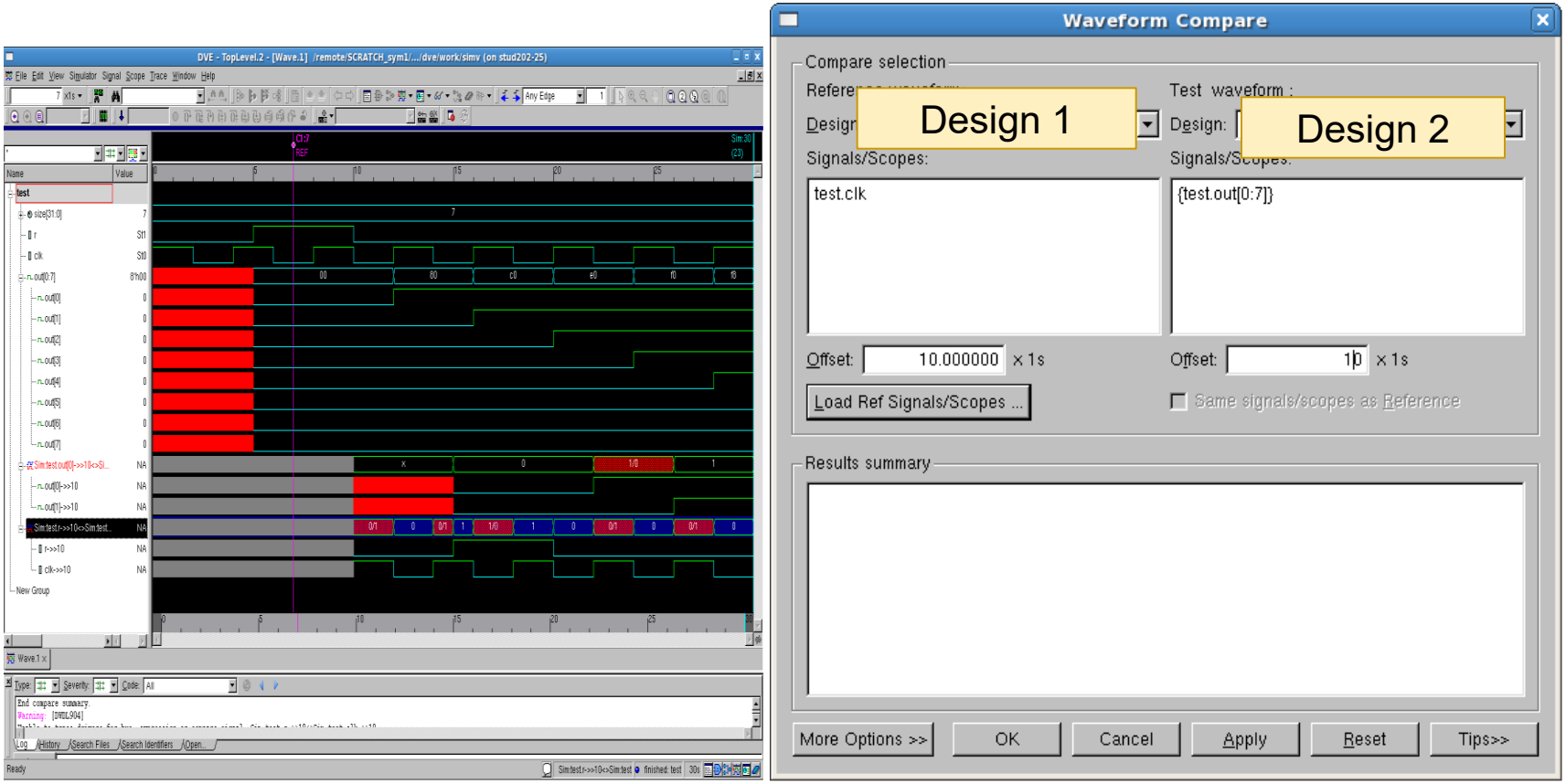


Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

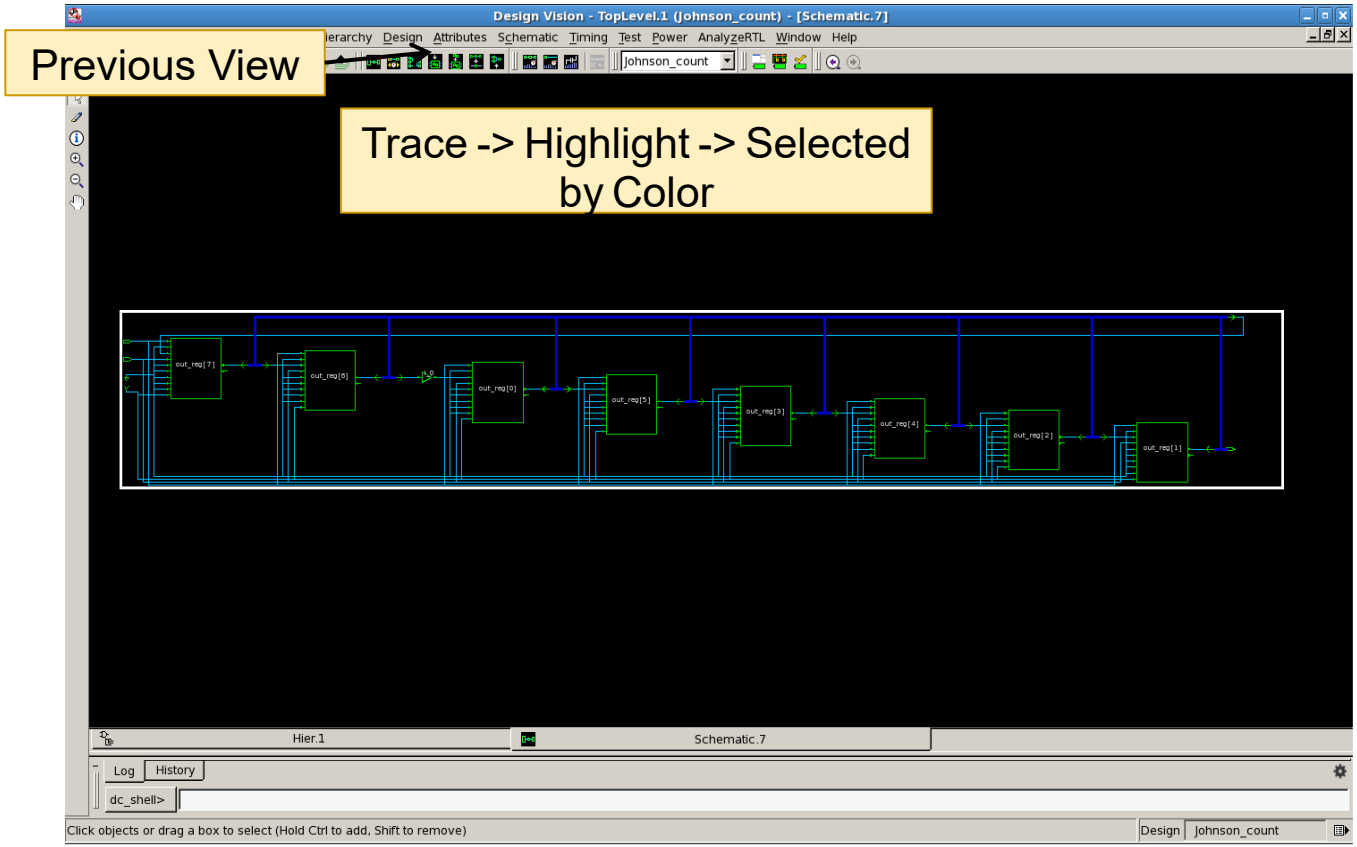
Tracing the Cause of Failed Assertion



RTL and Gate Signal Comparison



Gate Level Path Schematic

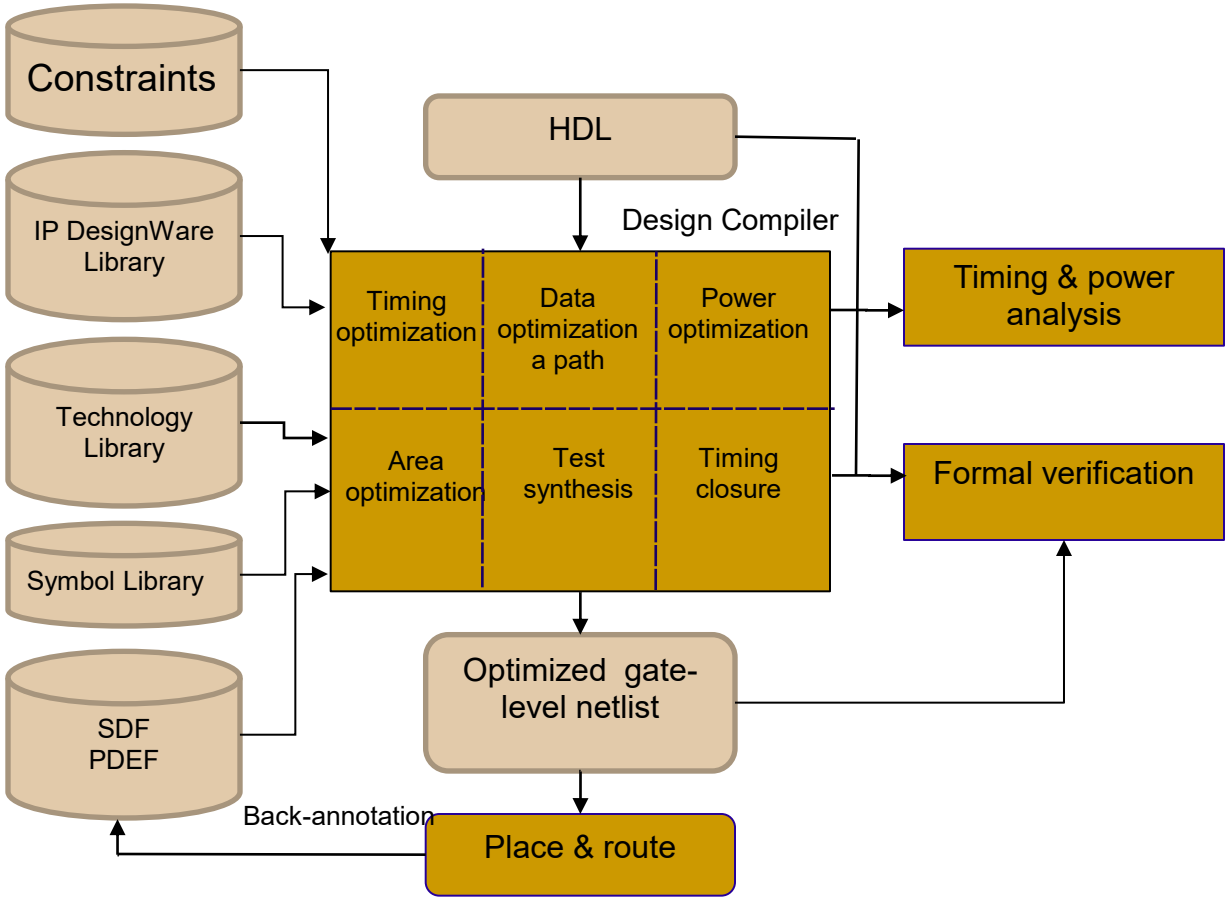


Logic Synthesis (Design Compiler)

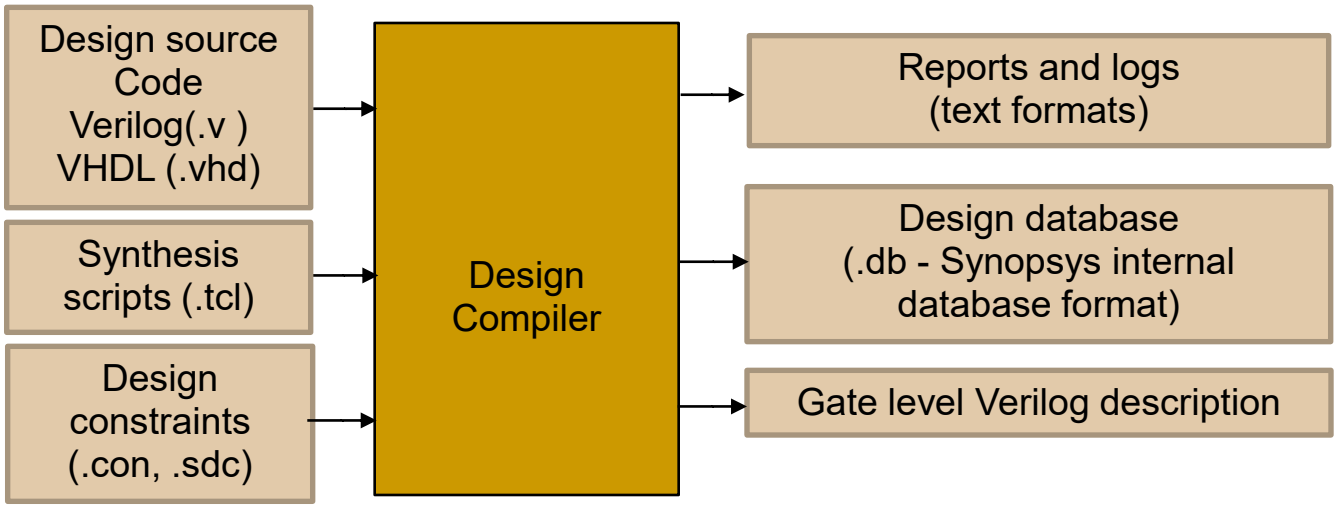
Introduction to Design Compiler

- Design Compiler performs logic synthesis and optimization of design
  - Synthesizes HDL designs into optimized technology-dependent gate-level designs.
  - Results in smallest and fastest logical representation of a given function
  - Design Compiler supports a wide range of flat and hierarchical design styles
  - Combinational and sequential designs can be optimized for:
    - Timing
    - Area
    - Power

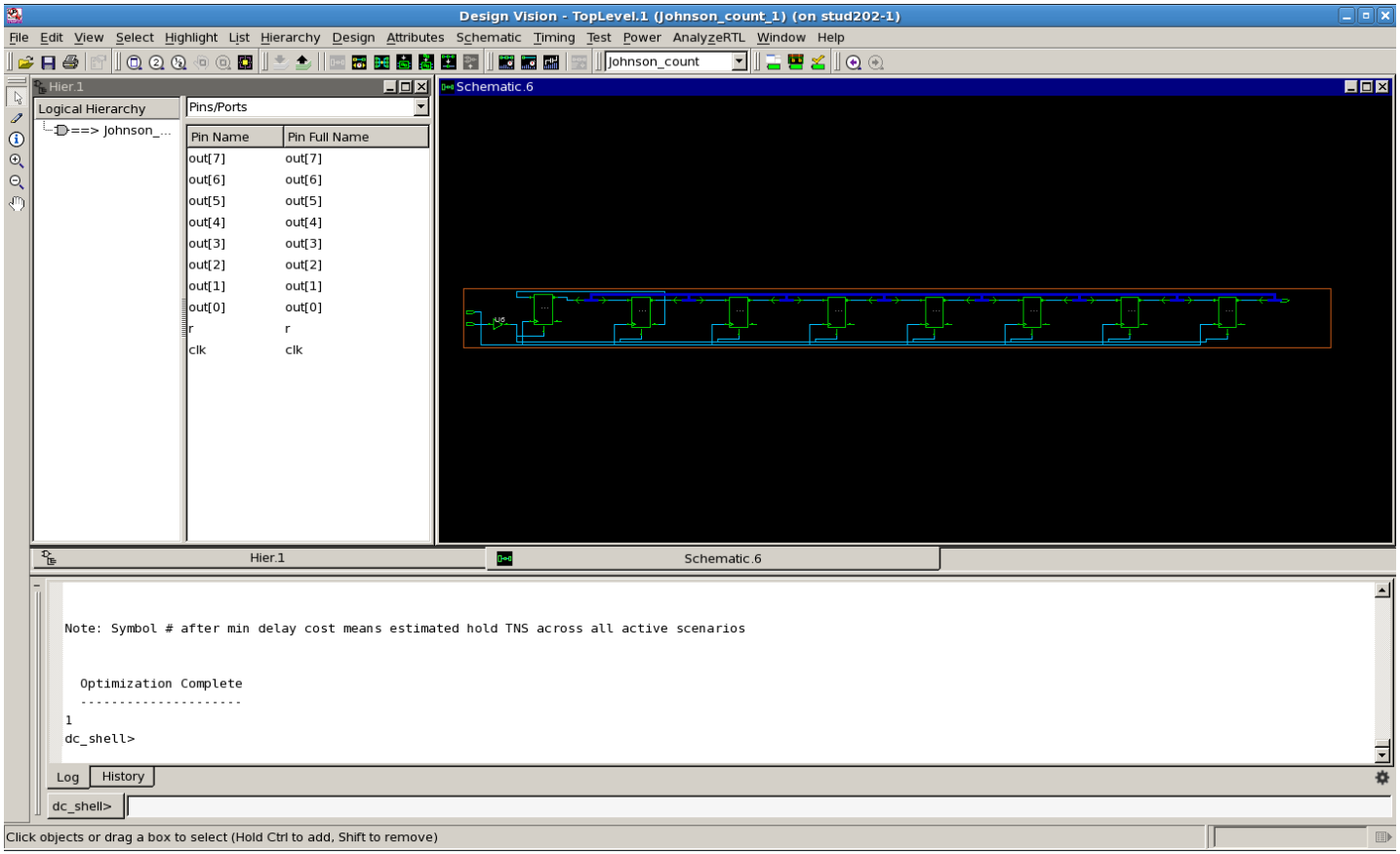
DC and Design Flow



Design Compiler Input and Output Files



Gate Level View After Compile



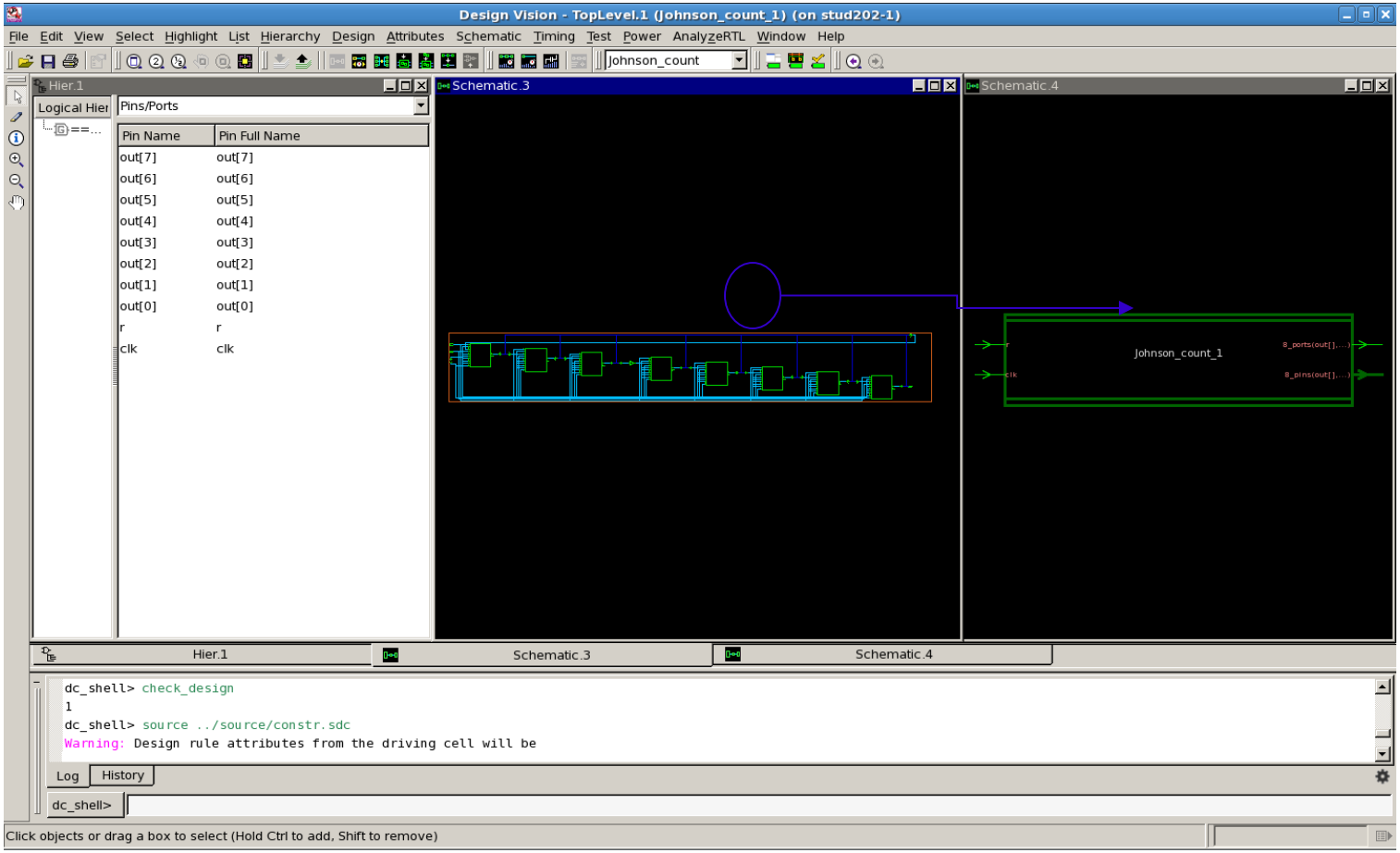
Design for Test

- Synopsys' design-for-test (DFT) synthesis solution (DFT Compiler) – enables scan insertion within Design Compiler
- DFT Compiler's integration with Design Compiler ensures DFT with optimization of area, power, and timing constraints, and predictable timing closure of physically optimized scan designs.

DFT Key Features

- One-pass test synthesis
- Comprehensive RTL and gate-level DFT design rule checking
- Rapid scan synthesis
- Adaptive scan technology
- Hierarchical scan synthesis
- Observe point insertion
- Automatic fixing of scan violations (autoFix)
- Location-based scan ordering
- Timing-based scan ordering

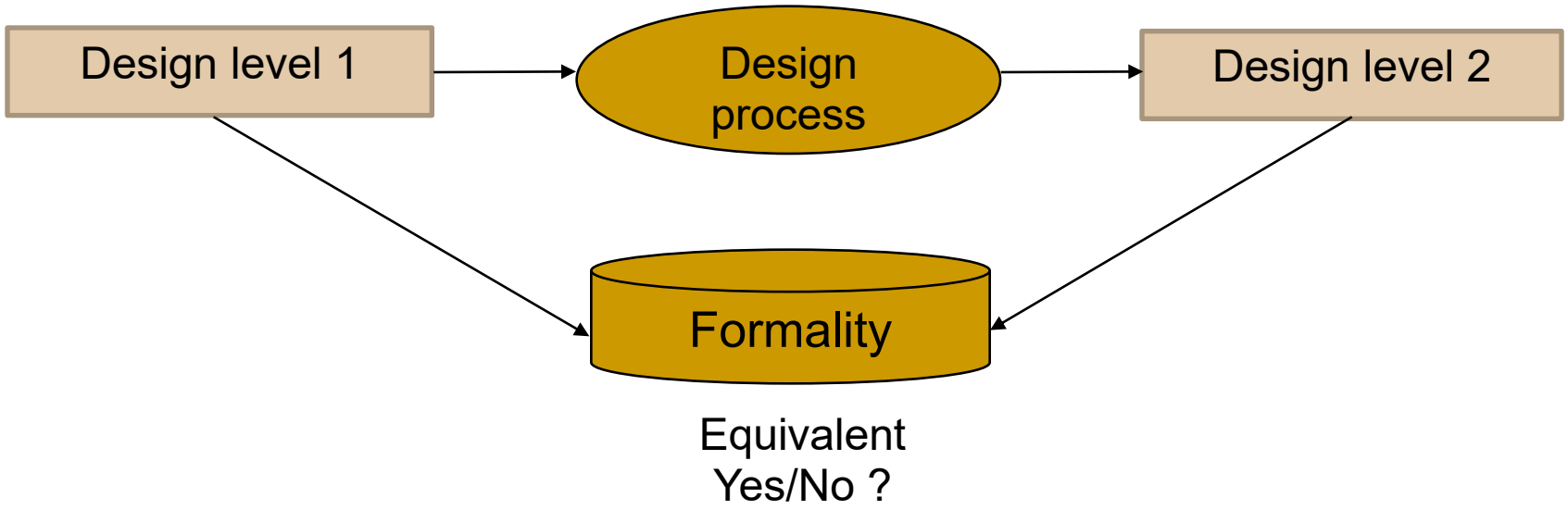
Schematic View After DFT



Formal Verification (Formality)

Formality Introduction

- Formality checks whether two designs are functionally equivalent or not
- Its purpose is to detect unexpected differences that may have been introduced into a design during development



Key Concepts

- Main concepts in Formality:
  - Compare Point
    - Primary output of a circuit
    - Registers within a circuit
    - An input to black boxes within circuit
  - Logic Cone
    - A block of combinational logic which drives a compare point

Equivalence Checking Verification Process

- Equivalence checking is a four-phase process:
- Reading and elaborating language descriptions into logical representations
- Setting up prompt for verification
- Mapping of corresponding compare points between pairs of designs (Matching)
- Comparison of logic cones that drive the compare points (Verification)

Input Files of Formality

- Formality supports the following input formats:

Input formats	Command
Verilog (synthesizable subset)	- read_verilog
Verilog (simulation libraries)	- read_verilog -vcs
VHDL (synthesizable subset)	- read_vhdl
EDIF	- read_edif
Synopsys binary files	- read_db, read_ddc, read_mdb (*)

Formality Flow Overview

- Guidance (Loading of automated setup file)
  - The purpose of automated file (.svf) is to help Formality process design changes caused by other tools, which it should have access to as the changes are made.
- Referencing (Specifying the reference design)
  - The reference design is the design against which the transformed (implementation) design is compared.
- Implementation (Specifying the implementation design)
  - This is the changed design. It is the design correctness that needs to be verified.
- Matching (Matching compare points)
  - Process of aligning compare points between two designs.
- Verification (Verify the Designs)
  - Process of proving or disproving that compare points are equivalent (have same functionality).
- Debug
  - During debug the user should determined where and why the comparison results were unsuccessful.

Automatic Test Pattern Generation (TetraMAX)

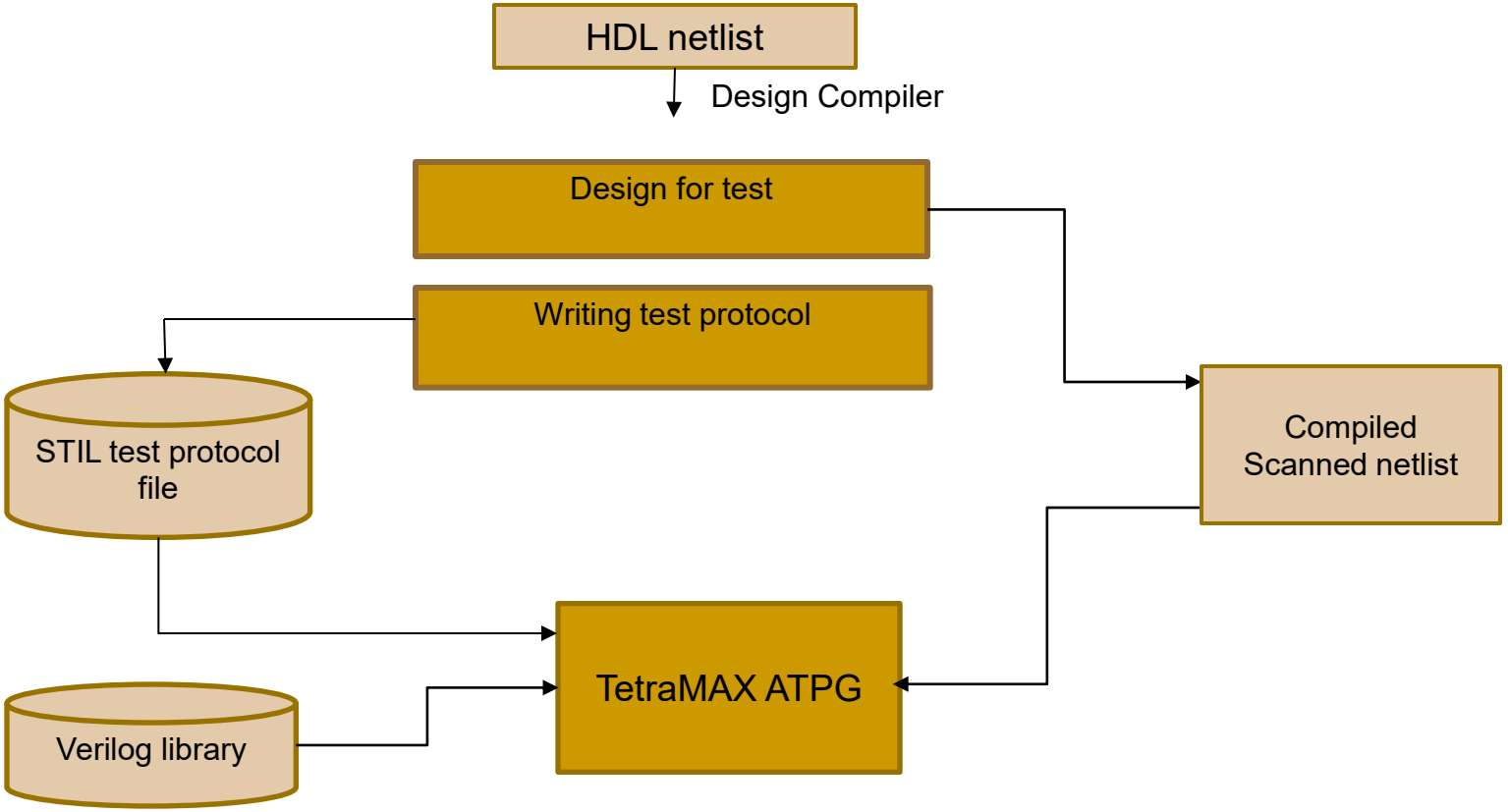
Introduction

- TetraMAX is a high-speed, high-capacity automatic test pattern generation (ATPG) tool
  - It can generate test patterns that maximize test coverage while using a minimum number of test vectors for a wide variety of design types and design flows.
  - It is well suited for designs of all sizes up to millions of gates

ATPG Modes

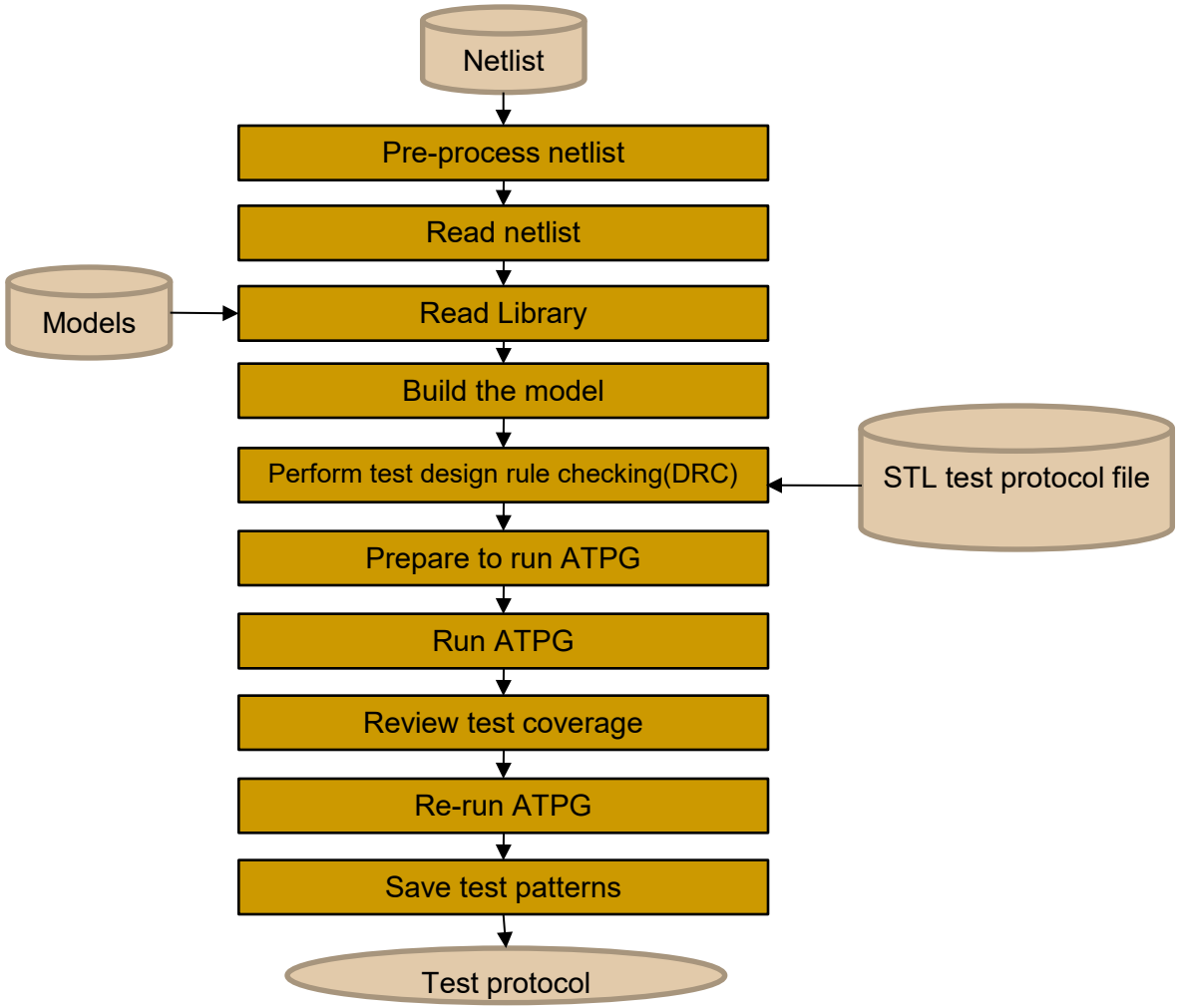
- TetraMAX offers a choice of ATPG modes:
  - Basic-Scan ATPG, an efficient combinational-only mode for full-scan designs
  - Fast-Sequential ATPG for limited support of partial-scan designs
  - Full-Sequential ATPG for maximum test coverage in partial-scan designs

Design Flow Using DFT Compiler and TetraMAX



Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

TetraMAX Design Flow



Steps of Running TetraMAX

- Reading the netlist
- Reading Verilog library models
- Building the ATPG model
- Performing Test Design Rule Checking (Test DRC)
- Generating test protocols

Test Design Rule Checking

- Test DRC checks for the following conditions:
  - Whether the scan chains inputs and outputs are logically connected
  - Whether all the clocks and asynchronous set/reset pins connected to scan chain flip-flops are controlled only by primary input ports
  - Whether the clocks/sets/resets are off when switching from normal mode to scan shift mode and again when switching back to normal mode
  - Whether any internal multiple-driver nets can be in contention

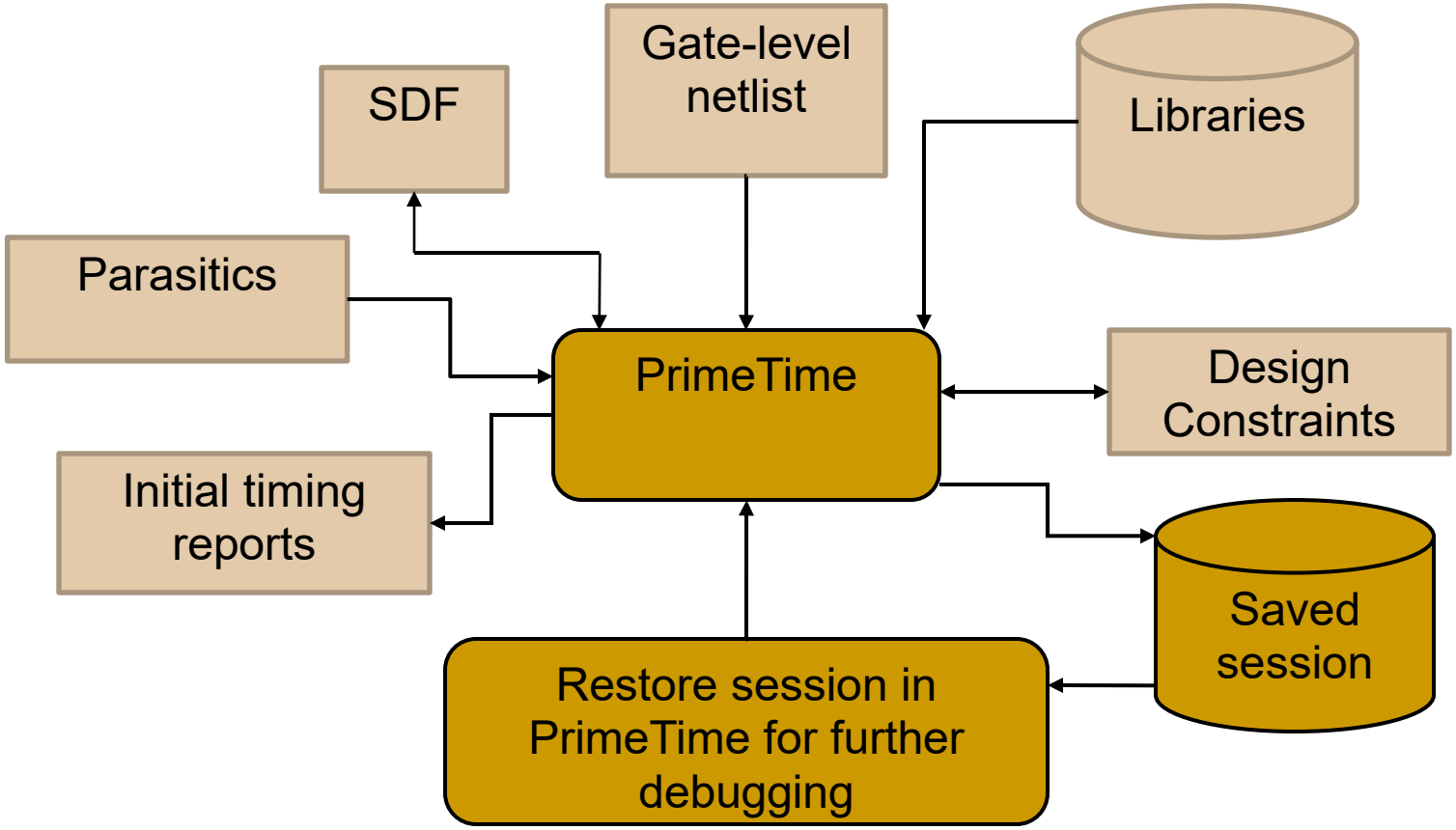
Static Timing Analysis (PrimeTime)

Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

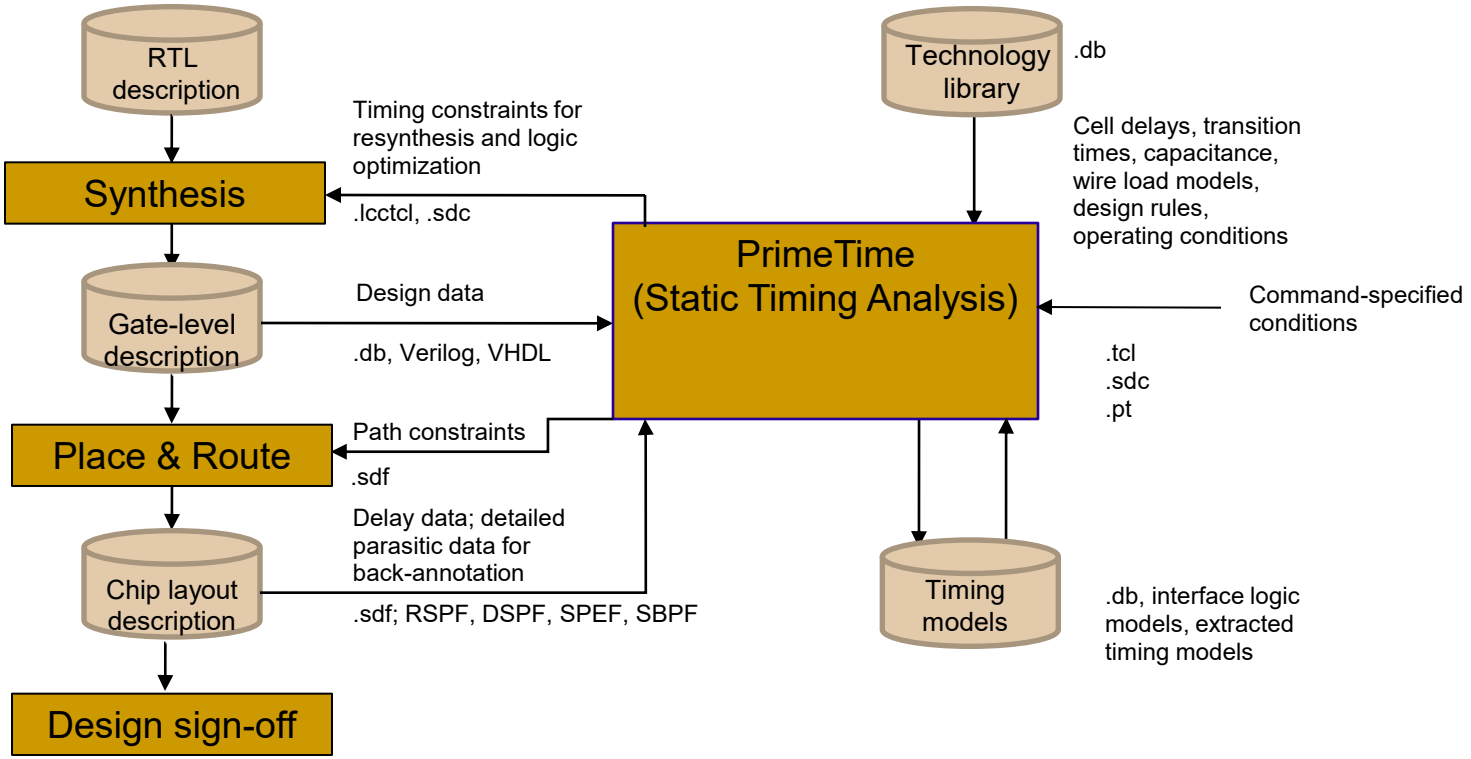
Introduction

- PrimeTime is a full-chip, gate-level static timing analysis tool that is an essential part of the design and analysis flow for today's large chip designs.
- PrimeTime validates the timing performance of a design by checking all possible paths for timing violations, without using logic simulation or test vectors.

PrimeTime Inputs and Outputs

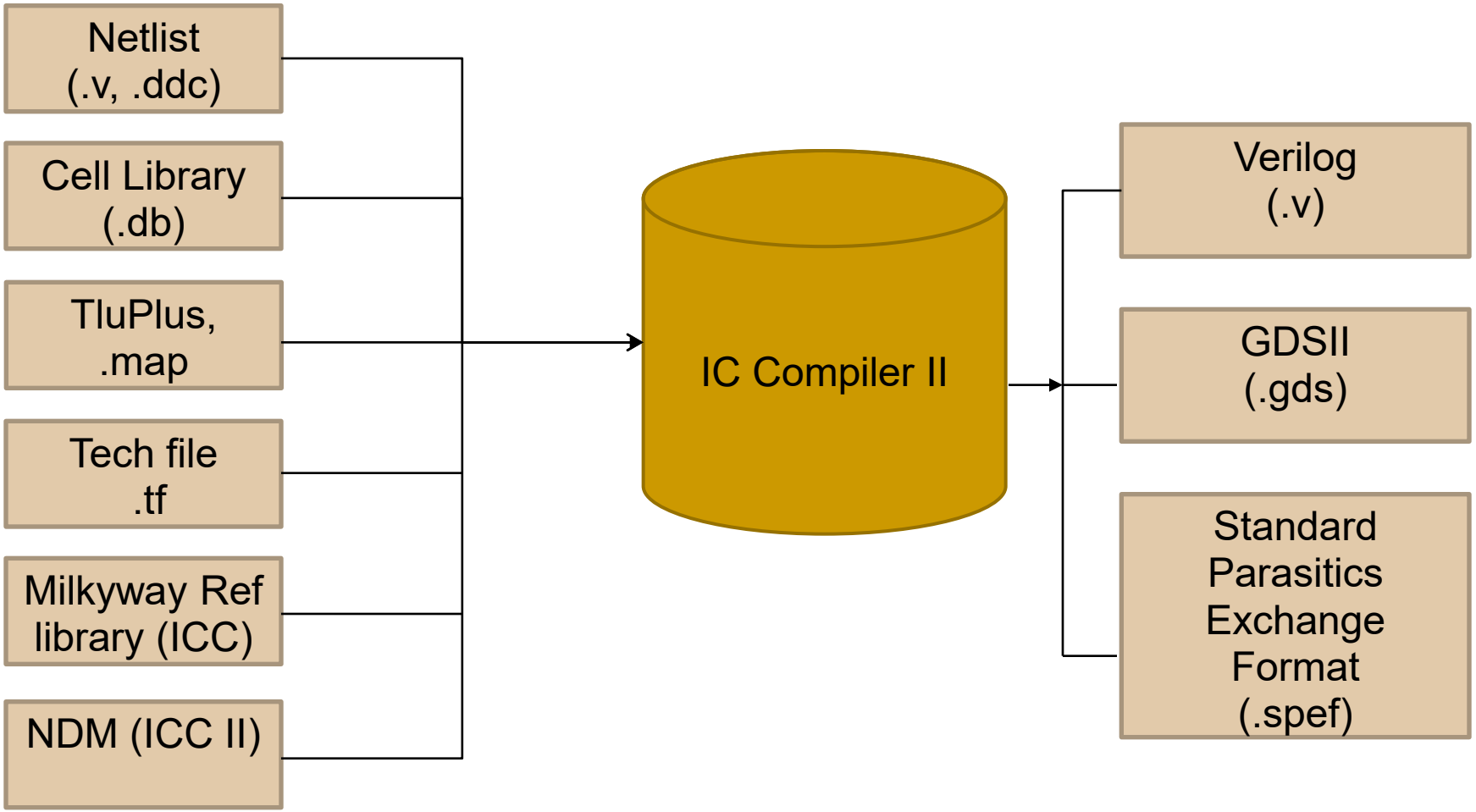


Using PrimeTime in Physical Synthesis Flow

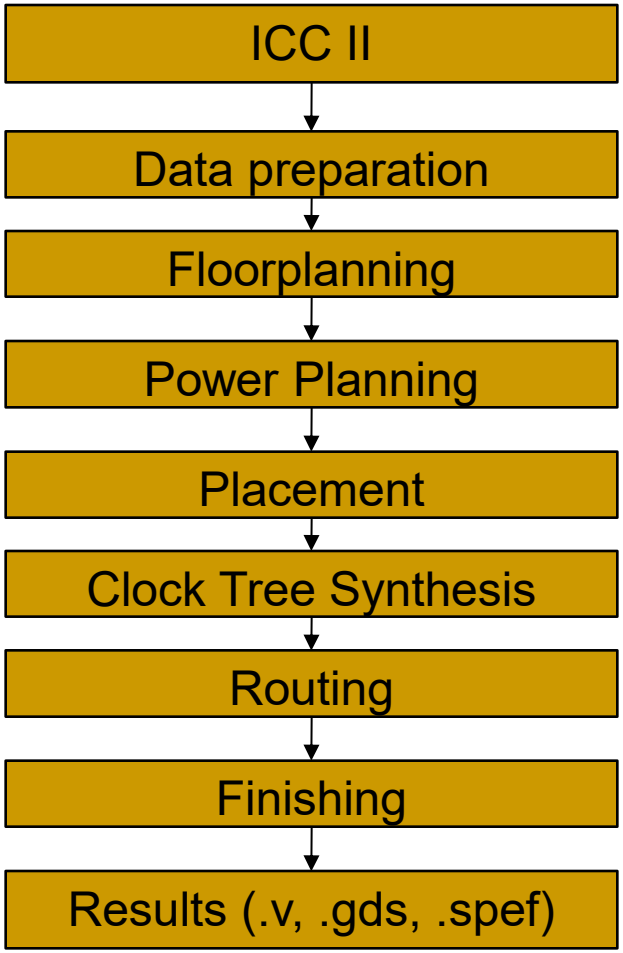


Physical Synthesis (IC Compiler II)

Input and Output Files of IC Compiler II



IC Compiler II Design Flow



Physical Design Steps

- Data preparation
  - Milkyway or new design library creation, logic libraries setup and parasitic models setup, design import.
- Floorplanning
  - Setting up the core area, top-level ports, and placement sites.
- Power Planning
  - Rectangular rings creation, power straps creation, etc.
- Core Placement and Optimization
  - During the placement phase the design's standard cells will be automatically placed in horizontal placement rows.
  - Allows following optimizations: power, optimize_dft, effort, etc.

Physical Design Steps (2)

- Core Clock Tree Synthesis and Optimization
  - During clock tree synthesis, IC Compiler II builds clock trees that meet the clock tree design rule constraints while balancing the loads and minimizing the clock skew. Allows the following options: area_recovery, power, optimize_dft, only_cts, etc.
- Core Routing and Optimization
  - This command performs simultaneous routing and postroute optimization on the current design. Allows following optimizations: power, size_only, stage, only_hold_time, only_design_rule, etc.
- Check DRC (Design Rule Checking) and LVS (layout vs. schematic)
- Export the design
  - Allows following formats: Verilog ,GDS format, etc.

Physical Verification (IC Validator)

Introduction

- IC Validator is a hierarchical physical verification tool that performs design rule checking (DRC) and layout vs. schematic (LVS) on IC design.

Input Files of IC Validator

- IC Validator uses several input files to perform DRC, LVS, LPE and ERC design verification. These input files are:
  - Database
  - Runset file
  - Schematic netlist
- The primary input file is the runset file.

Design Database

- In the beginning of a IC Validator run, primary group files are created that consist of one file per layer listed in the ASSIGN section of the runset file.
- Read different design databases using:
  - GDSII
  - GDSOUT
  - OASIS
  - Milkyway

Runset File

- Runset file is a control file that instructs IC Validator where to find input data, which checks to perform and where to write output files.
- Separate runsets are typically created for DRC and LVS.
- A DRC Runset instructs IC Validator to check layout files for errors.
  - A LVS Runset instructs IC Validator to compare the layout netlist to the schematic netlist of a design.

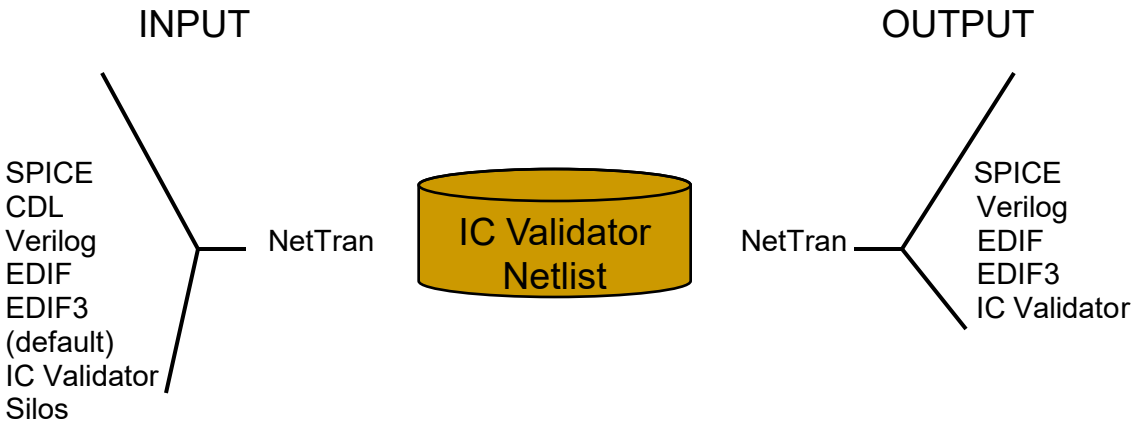
Schematic Netlist

- The schematic Netlist file is used during LVS comparison. It provides complete net information with each cell.
- If schematic netlist is in CDL, NetTran will translate it to IC Validator format.

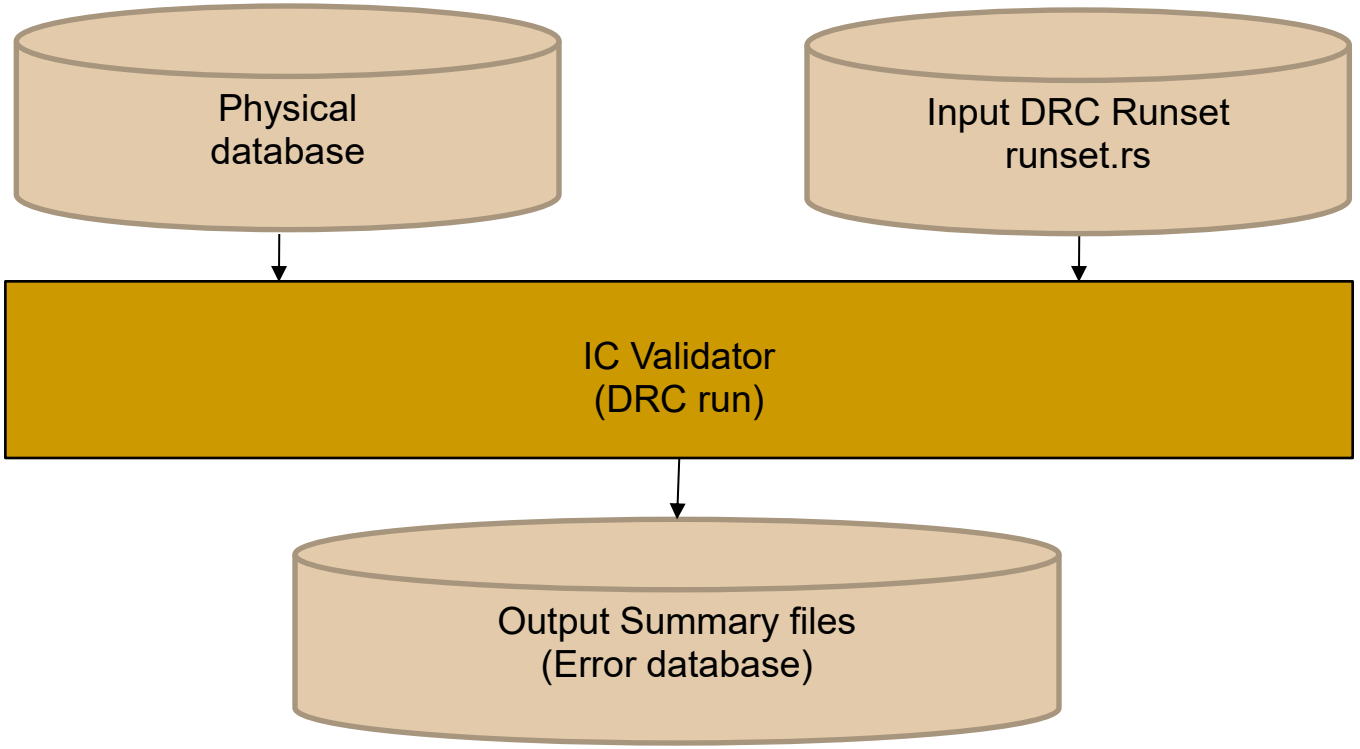
```
nettran –verilog Johnson_count.v –cdl-a-cdl-s-sp-S-verilog-b1 VDD –verilog-b0 VSS\
-rootCell Johnson_count –sp ./saed14nm.cdl –outName Johnson_count.sp
```


Netlist Translation

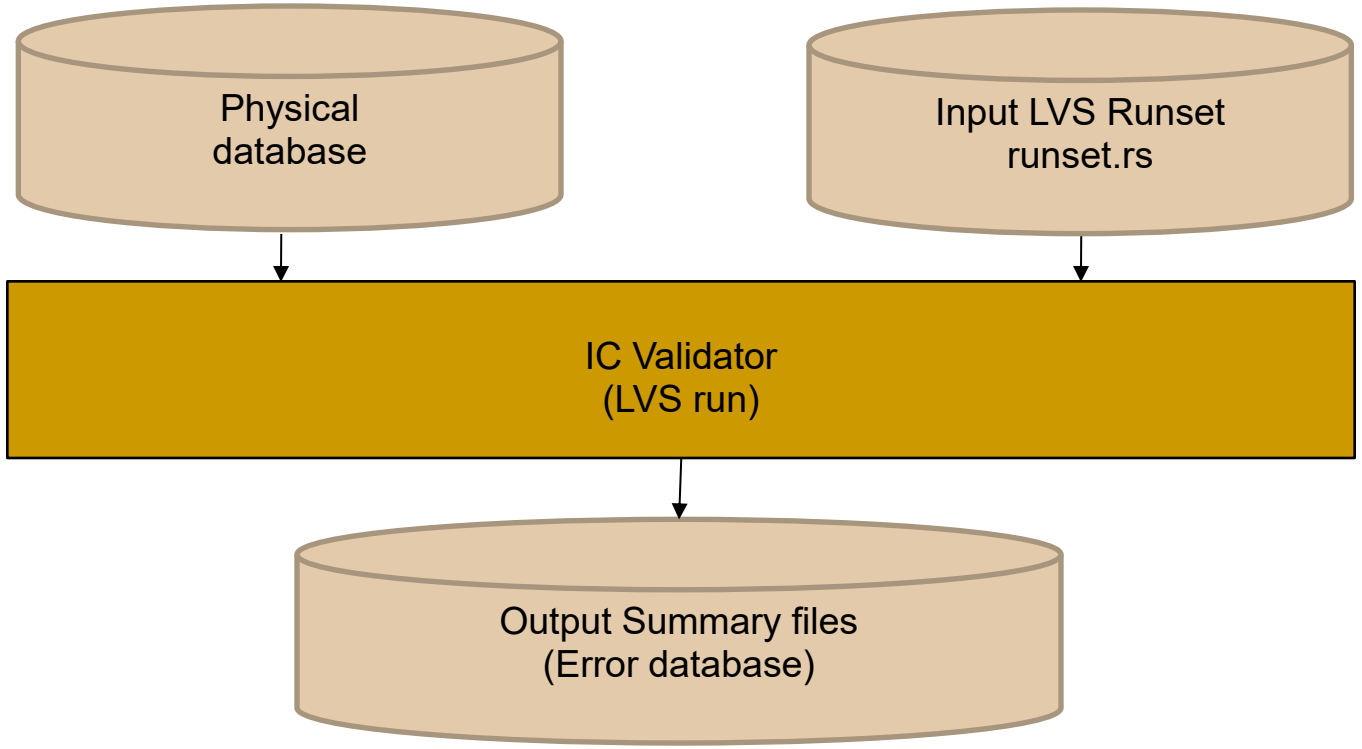
- IC Validator uses the NetTran utility to translate the netlist between different formats.
  (e.g. Verilog to SPICE, SPICE to IC Validator netlist format)



DRC Flow



LVS Flow



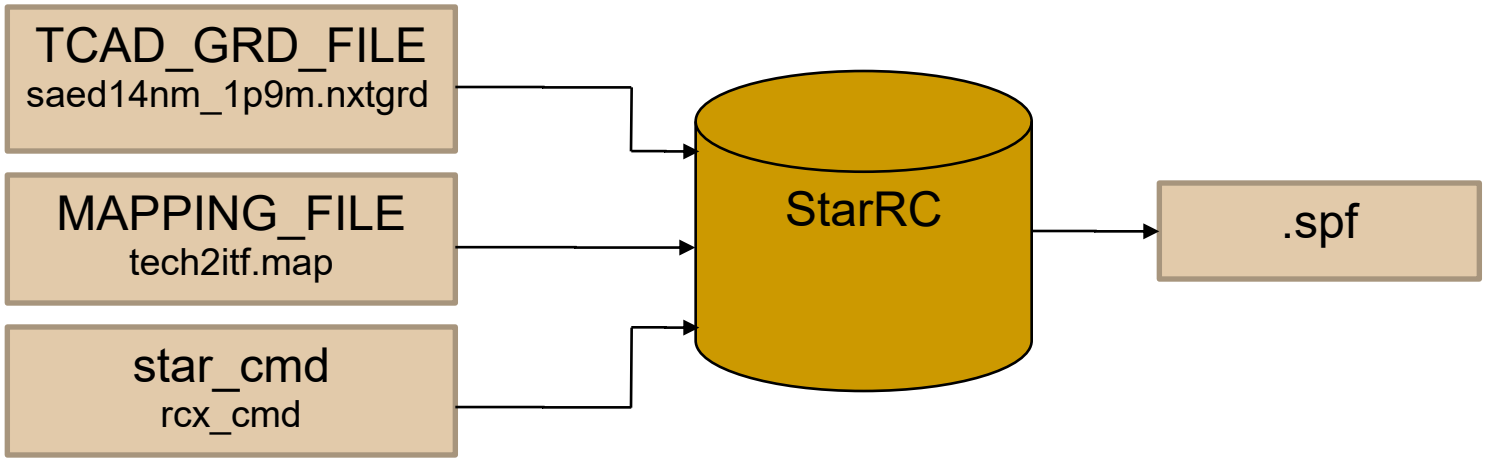
Layout Parasitics Extraction (StarRC)

Synopsys Design Flow Tutorial
Lecture - 1
Developed By: Vazgen Melikyan

StarRC Overview

- StarRC is a layout parasitic extraction tool. StarRC can be used at any physical design cycle stage to extract accurate parasitics.
- StarRC reads OpenAccess, Milkyway, LEF/DEF or IC Validator connected databases directly, without external processing.
- Extracted parasitics can be written into the Synopsys centralized Milkyway database for use by analysis and optimization tools.
- Because StarRC gracefully handles designs with layout versus schematic (LVS) violations, including opens and shorts, timing convergence can be ensured before the physical verification cycle begins.

Inputs and Outputs of StarRC



TCAD_GRD_FILE -File containing the modeled layers of a circuit.

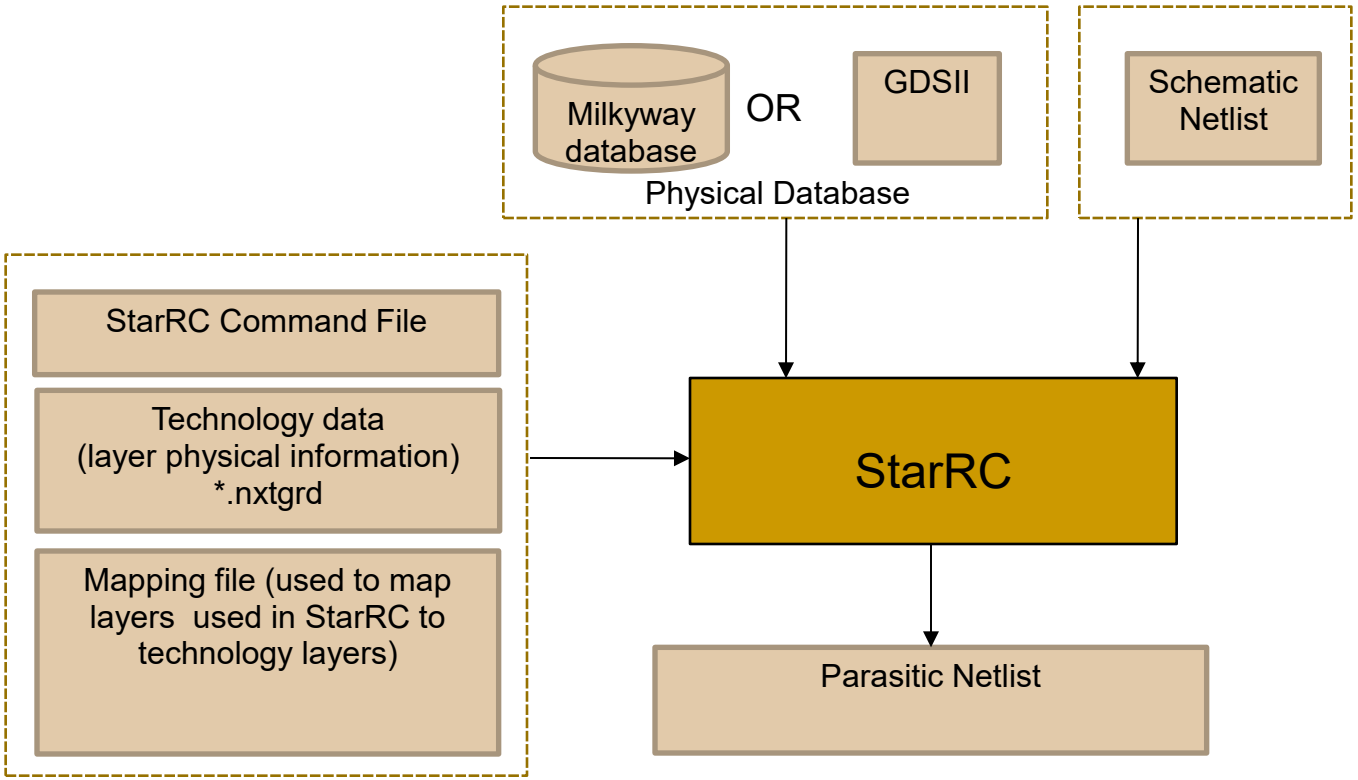
MAPPING_FILE-File containing physical layer mapping information between the input database and the specified TCAD_GRD_FILE

star_cmd -ASCII file containing StarRC commands that controls extraction functions

Input and Output Formats of StarRC

- StarRC supports these industry-standard formats:
  - Input formats
  - LEF(Layout Exchange Format)/DEF(Design Exchange Format)
  - GDSII
  - Milkyway
  - Output Netlist Formats
  - SPICE
  - Synopsys Binary Parasitic Format (SBPF)
  - Standard Parasitic Exchange Format (SPEF)
  - Detailed Standard Parasitic Format (DSPF)
  - StarRC accepts input from GDSII, LEF/DEF, and IC Compiler II formats

StarRC Extraction Flow

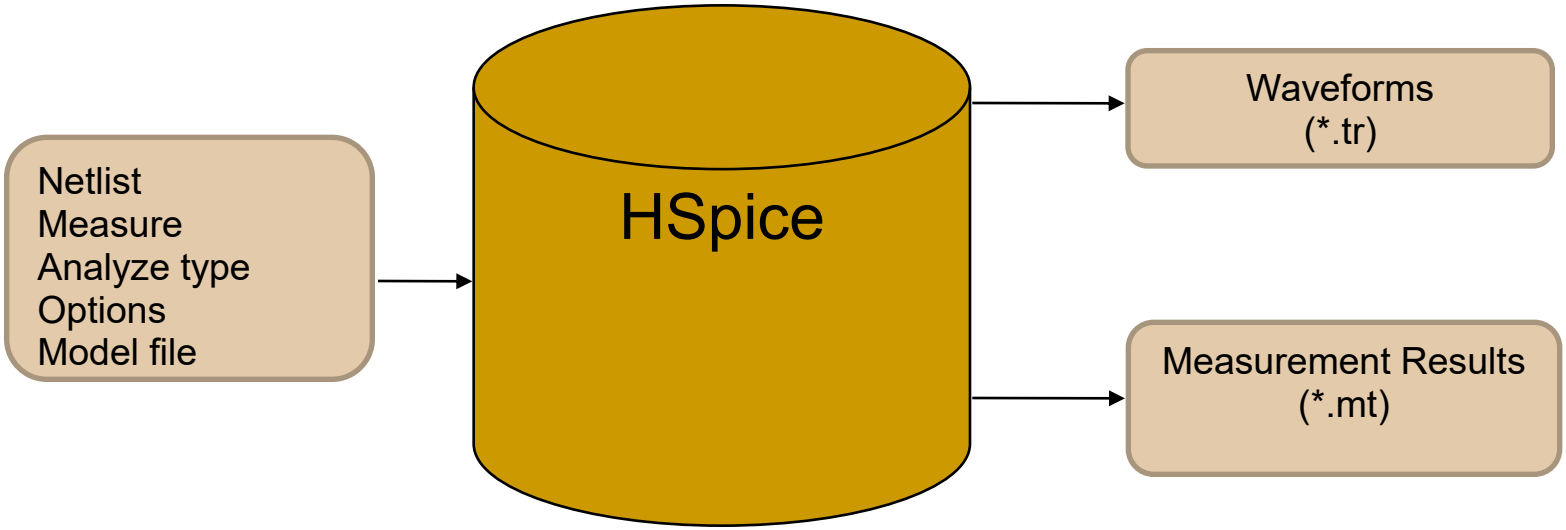


SPICE-Level Simulation (HSpice)

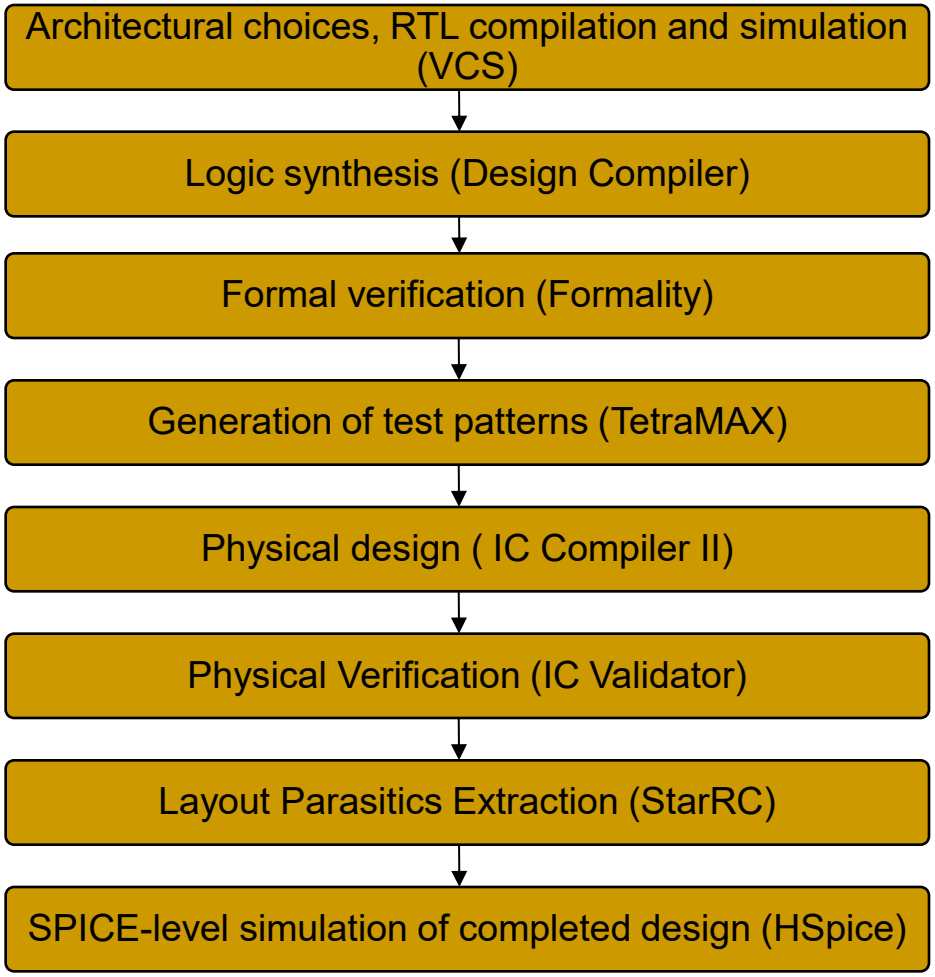
HSpice Features

- HSpice supports:
  - Analog/RF/mixed-signal IC Design
  - Verilog-A Behavioral Modeling
  - Design For Yield- Process Variability and MosRa Device Reliability Analysis
  - Transient Noise Analysis
  - Cell and Memory Characterization

Input and Output Files of HSpice



Synopsys Design Flow




SYNOPSYS®
Silicon to Software™